

SO(3) Equivariant Neural Networks

Lecture 1 Solutions

What Should Rotate?

Solutions to checkpoints, discussions, and notebook exercises

Phong Dang • prepared with AI assistance

1 Solutions to lecture questions

Question 1.1

Energy and distance are invariant scalars. Force and velocity are vector-equivariant. Stress is a rank-2 tensor and transforms as $T' = RTR^T$. These answers assume a proper rotation of the entire physical system, including all vector-valued inputs.

Question 1.2

Using the Euclidean dot product and orthogonality,

$$(Ru) \cdot (Rv) = (Ru)^T(Rv) = u^T R^T R v = u^T I v = u \cdot v.$$

The only property needed is $R^T R = I$.

Question 1.3

A scalar prediction can hide directional mistakes because its output does not need to carry orientation. A vector prediction must preserve directional information and satisfy $f(Rx) = Rf(x)$ component by component. It therefore tests both the invariant relationships in the inputs and the correct transformation of the output.

Question 1.4

No. Augmentation provides finitely many examples and encourages the model to interpolate consistently. It does not prove the identity for every input and every rotation. An architectural proof shows that the identity holds for all parameter values and group elements, apart from numerical roundoff.

Question 1.5

The factor $u \cdot v$ is invariant. The factor $u \times v$ is vector-equivariant under proper rotations. Hence

$$f(Ru, Rv) = ((Ru) \cdot (Rv))((Ru) \times (Rv)) = (u \cdot v)R(u \times v) = Rf(u, v).$$

An invariant scalar may safely rescale an equivariant vector.

Question 1.6

No. A fixed, unrotated laboratory field selects a preferred direction. Rotating only the other inputs does not describe a symmetry of the complete physical problem. Full rotational equivariance would be restored only if the field were also transformed, or the symmetry group were restricted to rotations that leave the field unchanged, namely rotations about its axis.

2 Notebook reference checks

For a proper rotation, the following residuals should be at floating-point scale:

```
orthogonality = np.max(np.abs(R.T @ R - np.eye(3)))
det_error = abs(np.linalg.det(R) - 1.0)
```

```
dot_error = abs((R @ u) @ (R @ v) - u @ v)
cross_error = np.max(np.abs(np.cross(R @ u, R @ v)
                                - R @ np.cross(u, v)))
```

A reusable relative equivariance error is

```
def equivariance_error(f, U, V, R):
    lhs = f(U @ R.T, V @ R.T)
    rhs = f(U, V) @ R.T
    return np.max(np.abs(lhs-rhs)) / max(np.max(np.abs(rhs)), 1e-12)
```

For the target $(u \cdot v)(u \times v)$, expect roughly 10^{-15} . For an axis-dependent function such as $f(u, v) = (u_x, v_y, u_z)$, expect a visible error because the coordinate axes are treated differently.

3 Exit-ticket answer

An equivariant model does not require its output to stay fixed; it requires the output to **transform according to the specified output representation**.